

```

//+-----
//|                                     RectangleZoneAnalyzer_02_v3.mq4
//|
//|  Блок 02: чтение zones.csv, поиск касаний и создание touches.csv
//|
//|  Касание подтверждается только закрытием свечи внутри зоны.
//|  Тени используются только для MFE/MAE.
//|  Будущие зоны при выборе структурных целей не используются.
//+-----
#property copyright "Copyright 2026"
#property version   "1.02"
#property strict
#property script_show_inputs

//--- Файлы находятся в MQL4\Files
input string InputZonesFileName    = "zones.csv";
input string OutputTouchesFileName = "touches.csv";

//--- ATR, полностью известный на момент касания
input ENUM_TIMEFRAMES TouchATRTimeframe = PERIOD_M5;
input int      TouchATRPeriod           = 14;

//--- Относительный тиковый объём
input int      RelativeVolumePeriod     = 20;

//--- Первичный локальный разворот
input double ReactionPoints             = 0.0;
input double ReactionATR                = 1.0;
input double ReactionZoneHeight         = 1.0;

//--- Локальный пробой после касания
input double LocalBreakoutPoints        = 10.0;
input double LocalBreakoutATR           = 0.10;

//--- Окна наблюдения
input int      MaxLocalResultBars       = 30;
input int      MaxStructuralResultBars  = 300;
input int      MFEMAEObservationBars    = 30;

//--- Печать прогресса
input int      ProgressEveryBars        = 5000;

#define ZONES_CSV_COLUMNS 63

```

```
//+-----  
//| Данные принятой зоны
```

```
//+-----  
struct ZoneData  
{  
    string    zoneID;  
    string    symbol;  
    string    timeframe;  
    string    zoneType;  
    string    finalStatus;  
  
    datetime  confirmBarTime;  
    datetime  confirmationTime;  
    datetime  brokenTime;  
    datetime  endTime;  
  
    double    zoneLow;  
    double    zoneHigh;  
    double    zoneHeightPrice;  
    double    zoneHeightPoints;  
    double    zoneHeightATR;  
  
    string    atrTimeframe;  
    long      atrPeriod;  
    double    atrValue;  
    double    zoneBreakoutThresholdPrice;  
  
    int       confirmationShift;  
  
    int       entryCount;  
    int       expectedTouchCount;  
    datetime  previousExpectedTouchTime;  
    int       previousExpectedTouchDecisionShift;  
  
    bool      active;  
};
```

```
//+-----  
//| Статистика свечи
```

```
//+-----  
struct BarStats  
{  
    double    open;  
    double    high;  
    double    low;  
    double    close;
```

```

double rangePrice;
double bodyPrice;
double upperWickPrice;
double lowerWickPrice;

double rangeATR;
double bodyATR;
double bodyPercent;
double upperWickPercent;
double lowerWickPercent;

long    tickVolume;
double  relativeVolume;
string  direction;
};

//+-----

//|  Запись одного входа цены в зону

//+-----
struct TouchRecord
{
    string touchID;
    string zoneID;
    string symbol;
    string timeframe;
    string zoneType;

    int    entryNumber;
    int    touchNumber;
    string touchStatus;
    string entryDirection;

    datetime touchBarTime;
    datetime touchDecisionTime;
    int      touchBarShift;
    int      touchDecisionShift;

    double zoneAgeCalendarMinutes;
    double zoneAgeCalendarHours;
    int    zoneAgeBars;
    double zoneAgeTradingHours;

    double minutesSincePreviousTouch;
    int    barsSincePreviousTouch;

    double zoneLow;
    double zoneHigh;
    double zoneHeightPrice;
    double zoneHeightPoints;

```

```

double zoneHeightATR;

double touchATR;
double touchClose;
double touchDepthPrice;
double touchDepthPoints;
double touchDepthPercent;

BarStats touchBar;
BarStats previousBar;

double approachGapPrice;
double approachGapPoints;
double approachGapATR;

double approachNetMoveATR3;
double approachNetMoveATR5;
double approachNetMoveATR10;

double approachRangeATR3;
double approachRangeATR5;
double approachRangeATR10;

int approachDirectionBars3;
int approachDirectionBars5;
int approachDirectionBars10;

int reversalTargetIndex;
string reversalTargetZoneID;
double reversalTargetLow;
double reversalTargetHigh;
double distanceToReversalTargetPrice;
double distanceToReversalTargetPoints;
double distanceToReversalTargetATR;
double reversalTargetAgeHours;
string reversalTargetStatus;

int breakoutTargetIndex;
string breakoutTargetZoneID;
double breakoutTargetLow;
double breakoutTargetHigh;
double distanceToBreakoutTargetPrice;
double distanceToBreakoutTargetPoints;
double distanceToBreakoutTargetATR;
double breakoutTargetAgeHours;
string breakoutTargetStatus;

double primaryReactionDistancePrice;
double primaryReactionDistancePoints;
double primaryReactionDistanceATR;
double primaryReactionDistanceZoneHeights;

```

```

double localBreakoutThresholdPrice;
double localBreakoutThresholdPoints;
double localBreakoutThresholdATR;

bool expectedExitReached;
datetime expectedExitTime;
int barsToExpectedExit;

bool reached05ATR;
bool reached10ATR;
bool reached15ATR;
bool reached20ATR;

bool reached1ZoneHeight;
bool reached2ZoneHeight;
bool reached3ZoneHeight;

bool primaryReversal;
datetime primaryReversalTime;
int barsToPrimaryReversal;

bool localBreakout;
datetime localBreakoutTime;
int barsToLocalBreakout;
double localBreakoutClose;
double localBreakoutDistancePoints;
double localBreakoutDistanceATR;

string localFirstResult;
string localSequence;

bool structuralReversal;
datetime structuralReversalTime;
int barsToStructuralReversal;
double structuralReversalMinutes;
double structuralReversalClose;

bool structuralBreakout;
datetime structuralBreakoutTime;
int barsToStructuralBreakout;
double structuralBreakoutMinutes;
double structuralBreakoutClose;

string structuralFirstResult;

double mfeClosePrice;
double mfeWickPrice;
double maeClosePrice;
double maeWickPrice;

datetime localObservationEndTime;
int localObservationBars;

```

```

    string localEndReason;

    datetime sequenceObservationEndTime;
    int sequenceObservationBars;
    string sequenceEndReason;

    datetime structuralObservationEndTime;
    int structuralObservationBars;
    string structuralEndReason;

    bool localFirstDone;
    bool localSequenceDone;
    bool structuralDone;
};

ZoneData    g_zones[];
TouchRecord g_touches[];

int g_zoneCount    = 0;
int g_touchCount   = 0;

int g_activeZoneIndexes[];
int g_activeZoneCount = 0;

int g_openTouchIndexes[];
int g_openTouchCount = 0;

int g_totalBars = 0;
int g_touchAttrTf = 0;
int g_nextZoneToActivate = 0;

int g_loadedRows           = 0;
int g_skippedRejected      = 0;
int g_skippedOtherChart    = 0;
int g_expectedTouches      = 0;
int g_wrongSideEntries     = 0;
int g_localReversalFirst   = 0;
int g_localBreakoutFirst   = 0;
int g_localTimeouts        = 0;
int g_structReversalFirst  = 0;
int g_structBreakoutFirst  = 0;
int g_structNoTargets       = 0;
int g_structTargetsBroken  = 0;
int g_structTimeouts       = 0;

//+-----

//| Служебные функции

//+-----
string TimeframeToString(int timeframe)

```

```

{
    int tf = timeframe;

    if(tf == PERIOD_CURRENT || tf == 0)
        tf = Period();

    switch(tf)
    {
        case PERIOD_M1: return("M1");
        case PERIOD_M5: return("M5");
        case PERIOD_M15: return("M15");
        case PERIOD_M30: return("M30");
        case PERIOD_H1: return("H1");
        case PERIOD_H4: return("H4");
        case PERIOD_D1: return("D1");
        case PERIOD_W1: return("W1");
        case PERIOD_MN1: return("MN1");
    }

    return("TF" + IntegerToString(tf));
}

string TimeToCsv(datetime value)
{
    if(value <= 0)
        return("");

    return(TimeToString(value, TIME_DATE | TIME_MINUTES));
}

string TimeToID(datetime value)
{
    string result = TimeToString(value, TIME_DATE | TIME_MINUTES);

    StringReplace(result, ".", "");
    StringReplace(result, ":", "");
    StringReplace(result, " ", "_");

    return(result);
}

string CsvField(string value)
{
    bool needQuotes = false;

    if(StringFind(value, ";") >= 0 ||
        StringFind(value, "\"") >= 0 ||
        StringFind(value, "\r") >= 0 ||
        StringFind(value, "\n") >= 0)
    {
        needQuotes = true;
    }
}

```

```

    }

    if(!needQuotes)
        return(value);

    StringReplace(value, "\"", "\\\"");
    return("\"" + value + "\"");
}

void AppendField(string &line, string value, bool first = false)
{
    if(!first)
        line += ";";

    line += CsvField(value);
}

string PriceToString(double value)
{
    return(DoubleToString(value, Digits));
}

string DoubleOrBlank(double value, int decimals = 6)
{
    if(value < 0.0)
        return("");

    return(DoubleToString(value, decimals));
}

string PriceOrBlank(double value)
{
    if(value < 0.0)
        return("");

    return(DoubleToString(value, Digits));
}

string IntOrBlank(int value)
{
    if(value < 0)
        return("");

    return(IntegerToString(value));
}

string BoolToCsv(bool value)
{
    return(value ? "1" : "0");
}

```

```

string LongToCsv(long value)
{
    return(DoubleToString((double)value, 0));
}

bool CloseInsideZone(double closePrice, double zoneLow, double
zoneHigh)
{
    return(closePrice >= zoneLow && closePrice <= zoneHigh);
}

bool IsBullZone(int zoneIndex)
{
    return(g_zones[zoneIndex].zoneType == "BULL");
}

//+-----
//| Плановое время закрытия бара
//+-----
datetime GetBarCloseTime(int timeframe, int shift)
{
    datetime barOpenTime = iTime(NULL, timeframe, shift);

    if(barOpenTime <= 0)
        return(0);

    int seconds = PeriodSeconds((ENUM_TIMEFRAMES)timeframe);

    if(seconds <= 0)
        return(0);

    return(barOpenTime + seconds);
}

//+-----
//| ATR, полностью известный к decisionTime
//+-----
double GetClosedATRAt(datetime decisionTime)
{
    if(TouchATRPeriod <= 0 || decisionTime <= 0)
        return(0.0);

    int atrBars = iBars(NULL, g_touchAtrTf);

    if(atrBars <= TouchATRPeriod + 2)
        return(0.0);
}

```

```

int atrShift = iBarShift(
    NULL,
    g_touchAtrTf,
    decisionTime,
    false
);

if(atrShift < 0)
    return(0.0);

/*
    Если decisionTime попал внутрь ещё не закрытого ATR-бара,
    используем предыдущий бар.

    Если decisionTime находится в торговом разрыве, iBarShift()
    возвращает последний существующий бар перед разрывом. Если его
    плановое время закрытия уже наступило, этот бар использовать не
    */
datetime atrBarCloseTime =
    GetBarCloseTime(g_touchAtrTf, atrShift);

if(atrBarCloseTime <= 0)
    return(0.0);

if(atrBarCloseTime > decisionTime)
    atrShift++;

if(atrShift >= atrBars)
    return(0.0);

double atrValue = iATR(
    NULL,
    g_touchAtrTf,
    TouchATRPeriod,
    atrShift
);

if(atrValue == EMPTY_VALUE || atrValue <= 0.0)
    return(0.0);

return(atrValue);
}

//+-----+
//| Относительный объём
//+-----+

```

```

double GetRelativeVolume(int shift, int period)
{
    if(period <= 0)
        return(-1.0);

    if(shift < 0 || shift >= g_totalBars)
        return(-1.0);

    if(shift + period >= g_totalBars)
        return(-1.0);

    double sum = 0.0;

    for(int i = 1; i <= period; i++)
        sum += (double)iVolume(NULL, 0, shift + i);

    double average = sum / period;

    if(average <= 0.0)
        return(-1.0);

    return((double)iVolume(NULL, 0, shift) / average);
}

```

//+-----

//| Статистика свечи

//+-----

```

void FillBarStats(int shift, double atrValue, BarStats &stats)
{
    stats.open    = iOpen(NULL, 0, shift);
    stats.high    = iHigh(NULL, 0, shift);
    stats.low     = iLow(NULL, 0, shift);
    stats.close   = iClose(NULL, 0, shift);

    stats.rangePrice = stats.high - stats.low;
    stats.bodyPrice  = MathAbs(stats.close - stats.open);

    stats.upperWickPrice =
        stats.high - MathMax(stats.open, stats.close);

    stats.lowerWickPrice =
        MathMin(stats.open, stats.close) - stats.low;

    if(stats.upperWickPrice < 0.0)
        stats.upperWickPrice = 0.0;

    if(stats.lowerWickPrice < 0.0)
        stats.lowerWickPrice = 0.0;
}

```

```

if(atrValue > 0.0)
{
    stats.rangeATR = stats.rangePrice / atrValue;
    stats.bodyATR  = stats.bodyPrice / atrValue;
}
else
{
    stats.rangeATR = -1.0;
    stats.bodyATR  = -1.0;
}

if(stats.rangePrice > 0.0)
{
    stats.bodyPercent =
        100.0 * stats.bodyPrice / stats.rangePrice;

    stats.upperWickPercent =
        100.0 * stats.upperWickPrice / stats.rangePrice;

    stats.lowerWickPercent =
        100.0 * stats.lowerWickPrice / stats.rangePrice;
}
else
{
    stats.bodyPercent      = 0.0;
    stats.upperWickPercent = 0.0;
    stats.lowerWickPercent = 0.0;
}

stats.tickVolume = iVolume(NULL, 0, shift);
stats.relativeVolume =
    GetRelativeVolume(shift, RelativeVolumePeriod);

double difference = stats.close - stats.open;

if(MathAbs(difference) < Point * 0.5)
    stats.direction = "DOJI";
else if(difference > 0.0)
    stats.direction = "BULLISH";
else
    stats.direction = "BEARISH";
}

//+-----
//| Расчёт параметров подхода
//+-----
void CalculateApproachWindow(
    int touchShift,
    int windowBars,

```

```

bool bullishZone,
double atrValue,
double &netMoveATR,
double &rangeATR,
int &directionBars
)
{
    netMoveATR    = -1.0;
    rangeATR      = -1.0;
    directionBars = -1;

    if(windowBars <= 0)
        return;

    if(touchShift + windowBars >= g_totalBars)
        return;

    double touchClose = iClose(NULL, 0, touchShift);
    double startClose = iClose(NULL, 0, touchShift + windowBars
);

    double netMovePrice;

    if(bullishZone)
        netMovePrice = startClose - touchClose;
    else
        netMovePrice = touchClose - startClose;

    if(atrValue > 0.0)
        netMoveATR = netMovePrice / atrValue;

    double highest = -1.0e100;
    double lowest  = 1.0e100;
    int countTowardZone = 0;

    // Окно содержит windowBars свечей и заканчивается свечой касания.
    for(int j = 0; j < windowBars; j++)
    {
        int shift = touchShift + j;

        double high  = iHigh(NULL, 0, shift);
        double low   = iLow(NULL, 0, shift);
        double open  = iOpen(NULL, 0, shift);
        double close = iClose(NULL, 0, shift);

        if(high > highest)
            highest = high;

        if(low < lowest)
            lowest = low;
    }
}

```

```

        if(bullishZone)
        {
            if(close < open)
                countTowardZone++;
        }
        else
        {
            if(close > open)
                countTowardZone++;
        }
    }

    if(atrValue > 0.0)
        rangeATR = (highest - lowest) / atrValue;

    directionBars = countTowardZone;
}

```

//+-----

//| Чтение zones.csv

//+-----

```

bool LoadZonesCsv()
{
    ResetLastError();

    int handle = FileOpen(
        InputZonesFileName,
        FILE_READ | FILE_CSV | FILE_ANSI,
        ';'
    );

    if(handle == INVALID_HANDLE)
    {
        Print(
            "Ошибка открытия ",
            InputZonesFileName,
            ". Код ошибки: ",
            GetLastError()
        );

        ResetLastError();
        return(false);
    }

    string fields[];
    ArrayResize(fields, ZONES_CSV_COLUMNS);

    // Пропускаем заголовок.
    for(int h = 0; h < ZONES_CSV_COLUMNS; h++)

```

```

{
    if(FileIsEnding(handle))
    {
        Print(
"Ошибка: zones.csv не содержит полного заголовка.");
        FileClose(handle);
        return(false);
    }

    fields[h] = FileReadString(handle);
}

datetime previousConfirmation = 0;

while(!FileIsEnding(handle))
{
    for(int c = 0; c < ZONES_CSV_COLUMNS; c++)
    {
        if(FileIsEnding(handle) && c == 0)
        {
            fields[c] = "";
            break;
        }

        fields[c] = FileReadString(handle);
    }

    if(StringLen(fields[0]) == 0)
        break;

    g_loadedRows++;

    if(fields[4] != "ACCEPTED")
    {
        g_skippedRejected++;
        continue;
    }

    if(fields[1] != Symbol() ||
        fields[2] != TimeframeToString(Period()))
    {
        g_skippedOtherChart++;
        continue;
    }

    int index = g_zoneCount;
    ArrayResize(g_zones, g_zoneCount + 1);
    g_zoneCount++;

    g_zones[index].zoneID      = fields[0];
    g_zones[index].symbol      = fields[1];
    g_zones[index].timeframe   = fields[2];

```

```

g_zones[index].zoneType      = fields[3];
g_zones[index].finalStatus   = fields[13];

g_zones[index].confirmBarTime    = StringToTime(fields[8
]);
g_zones[index].confirmationTime = StringToTime(fields[9
]);
g_zones[index].brokenTime =
    StringLen(fields[11]) > 0 ? StringToTime(fields[11]) :
0;
g_zones[index].endTime =
    StringLen(fields[12]) > 0 ? StringToTime(fields[12]) :
0;

g_zones[index].zoneLow          = StringToDouble(fields[14
]);
g_zones[index].zoneHigh         = StringToDouble(fields[15
]);
g_zones[index].zoneHeightPrice = StringToDouble(fields[16
]);
g_zones[index].zoneHeightPoints = StringToDouble(fields[
17]);
g_zones[index].zoneHeightATR    =
    StringLen(fields[18]) > 0 ? StringToDouble(fields[18])
: -1.0;

g_zones[index].atrTimeframe = fields[19];
g_zones[index].atrPeriod    = StringToInteger(fields[20
]);
g_zones[index].atrValue     = StringToDouble(fields[21]);

g_zones[index].zoneBreakoutThresholdPrice =
    StringToDouble(fields[30]);

int confirmBarShift =
    iBarShift(NULL, 0, g_zones[index].confirmBarTime, true
);

/*
confirmationShift — это индекс решения после закрытия
подтверждающего бара. Он равен индексу следующего реально
существующего бара, поэтому корректно считает торговые бары
даже через ночные и выходные разрывы.
*/
g_zones[index].confirmationShift = confirmBarShift - 1;

g_zones[index].entryCount = 0;
g_zones[index].expectedTouchCount = 0;
g_zones[index].previousExpectedTouchTime = 0;
g_zones[index].previousExpectedTouchDecisionShift = -1;

```

```

g_zones[index].active = false;

if(g_zones[index].confirmBarTime <= 0 ||
    g_zones[index].confirmationTime <= 0 ||
    confirmBarShift <= 0 ||
    g_zones[index].confirmationShift < 0 ||
    g_zones[index].zoneHigh <= g_zones[index].zoneLow)
{
    Print("Ошибка данных зоны: ", g_zones[index].zoneID);
    FileClose(handle);
    return(false);
}

if(previousConfirmation > 0 &&
    g_zones[index].confirmationTime < previousConfirmation
)
{
    Print(
"Ошибка: принятые зоны в zones.csv расположены не по времени. Зона:
",
        g_zones[index].zoneID
    );

    FileClose(handle);
    return(false);
}

previousConfirmation = g_zones[index].confirmationTime;
}

FileClose(handle);

if(g_zoneCount <= 0)
{
    Print(
"В zones.csv не найдено принятых зон для текущего графика.");
    return(false);
}

return(true);
}

//+-----

//| АКТИВНЫЕ ЗОНЫ

//+-----

void AddActiveZone(int zoneIndex)
{
    if(g_zones[zoneIndex].active)
        return;

```

```

    ArrayResize(g_activeZoneIndexes, g_activeZoneCount + 1);
    g_activeZoneIndexes[g_activeZoneCount] = zoneIndex;
    g_activeZoneCount++;
    g_zones[zoneIndex].active = true;
}

void RemoveActiveZonePosition(int position)
{
    if(position < 0 || position >= g_activeZoneCount)
        return;

    int zoneIndex = g_activeZoneIndexes[position];
    g_zones[zoneIndex].active = false;

    for(int i = position; i < g_activeZoneCount - 1; i++)
        g_activeZoneIndexes[i] = g_activeZoneIndexes[i + 1];

    g_activeZoneCount--;
    ArrayResize(g_activeZoneIndexes, g_activeZoneCount);
}

void ActivateKnownZones(datetime decisionTime)
{
    while(g_nextZoneToActivate < g_zoneCount &&
        g_zones[g_nextZoneToActivate].confirmationTime <
decisionTime)
    {
        AddActiveZone(g_nextZoneToActivate);
        g_nextZoneToActivate++;
    }
}

void RemoveBrokenZones(datetime decisionTime)
{
    int position = 0;

    while(position < g_activeZoneCount)
    {
        int zoneIndex = g_activeZoneIndexes[position];

        if(g_zones[zoneIndex].brokenTime > 0 &&
            g_zones[zoneIndex].brokenTime <= decisionTime)
        {
            RemoveActiveZonePosition(position);
            continue;
        }

        position++;
    }
}

```

```

//+-----
//| Выбор структурных целей
//+-----
void SelectStructuralTargets(
    int sourceZoneIndex,
    datetime touchDecisionTime,
    int &reversalTargetIndex,
    int &breakoutTargetIndex
)
{
    reversalTargetIndex = -1;
    breakoutTargetIndex = -1;

    double nearestReversalDistance = 1.0e100;
    double nearestBreakoutDistance = 1.0e100;

    bool sourceBull = IsBullZone(sourceZoneIndex);

    for(int i = 0; i < g_activeZoneCount; i++)
    {
        int candidateIndex = g_activeZoneIndexes[i];

        if(candidateIndex == sourceZoneIndex)
            continue;

        if(g_zones[candidateIndex].confirmationTime >=
touchDecisionTime)
            continue;

        if(g_zones[candidateIndex].brokenTime > 0 &&
            g_zones[candidateIndex].brokenTime <=
touchDecisionTime)
        {
            continue;
        }

        bool candidateBull = IsBullZone(candidateIndex);
        double distance = 0.0;

        if(sourceBull)
        {
            // Разворот вверх: ближайшая BEAR-зона полностью выше.
            if(!candidateBull &&
                g_zones[candidateIndex].zoneLow >=
                g_zones[sourceZoneIndex].zoneHigh)
            {
                distance =
                    g_zones[candidateIndex].zoneLow -
                    g_zones[sourceZoneIndex].zoneHigh;
            }
        }
    }
}

```

```

        if(distance < nearestReversalDistance)
        {
            nearestReversalDistance = distance;
            reversalTargetIndex = candidateIndex;
        }
    }

    // Пробой вниз: ближайшая более низкая BULL-зона.
    if(candidateBull &&
        g_zones[candidateIndex].zoneHigh <=
        g_zones[sourceZoneIndex].zoneLow)
    {
        distance =
            g_zones[sourceZoneIndex].zoneLow -
            g_zones[candidateIndex].zoneHigh;

        if(distance < nearestBreakoutDistance)
        {
            nearestBreakoutDistance = distance;
            breakoutTargetIndex = candidateIndex;
        }
    }
}
else
{
    // Разворот вниз: ближайшая BULL-зона полностью ниже.
    if(candidateBull &&
        g_zones[candidateIndex].zoneHigh <=
        g_zones[sourceZoneIndex].zoneLow)
    {
        distance =
            g_zones[sourceZoneIndex].zoneLow -
            g_zones[candidateIndex].zoneHigh;

        if(distance < nearestReversalDistance)
        {
            nearestReversalDistance = distance;
            reversalTargetIndex = candidateIndex;
        }
    }

    // Пробой вверх: ближайшая более высокая BEAR-зона.
    if(!candidateBull &&
        g_zones[candidateIndex].zoneLow >=
        g_zones[sourceZoneIndex].zoneHigh)
    {
        distance =
            g_zones[candidateIndex].zoneLow -
            g_zones[sourceZoneIndex].zoneHigh;

        if(distance < nearestBreakoutDistance)
        {

```

```

        nearestBreakoutDistance = distance;
        breakoutTargetIndex = candidateIndex;
    }
}
}
}

void FillTargetData(
    int targetIndex,
    int sourceZoneIndex,
    datetime touchDecisionTime,
    double touchATR,
    bool reversalTarget,
    TouchRecord &touch
)
{
    if(targetIndex < 0)
    {
        if(reversalTarget)
        {
            touch.reversalTargetIndex = -1;
            touch.reversalTargetZoneID = "NONE";
            touch.reversalTargetLow = -1.0;
            touch.reversalTargetHigh = -1.0;
            touch.distanceToReversalTargetPrice = -1.0;
            touch.distanceToReversalTargetPoints = -1.0;
            touch.distanceToReversalTargetATR = -1.0;
            touch.reversalTargetAgeHours = -1.0;
            touch.reversalTargetStatus = "NONE";
        }
        else
        {
            touch.breakoutTargetIndex = -1;
            touch.breakoutTargetZoneID = "NONE";
            touch.breakoutTargetLow = -1.0;
            touch.breakoutTargetHigh = -1.0;
            touch.distanceToBreakoutTargetPrice = -1.0;
            touch.distanceToBreakoutTargetPoints = -1.0;
            touch.distanceToBreakoutTargetATR = -1.0;
            touch.breakoutTargetAgeHours = -1.0;
            touch.breakoutTargetStatus = "NONE";
        }

        return;
    }

    bool sourceBull = IsBullZone(sourceZoneIndex);
    double distancePrice;

    if(sourceBull)
    {

```

```

    if(reversalTarget)
    {
        distancePrice =
            g_zones[targetIndex].zoneLow -
            g_zones[sourceZoneIndex].zoneHigh;
    }
    else
    {
        distancePrice =
            g_zones[sourceZoneIndex].zoneLow -
            g_zones[targetIndex].zoneHigh;
    }
}
else
{
    if(reversalTarget)
    {
        distancePrice =
            g_zones[sourceZoneIndex].zoneLow -
            g_zones[targetIndex].zoneHigh;
    }
    else
    {
        distancePrice =
            g_zones[targetIndex].zoneLow -
            g_zones[sourceZoneIndex].zoneHigh;
    }
}

if(distancePrice < 0.0)
    distancePrice = 0.0;

double ageHours =
    (double)(touchDecisionTime -
        g_zones[targetIndex].confirmationTime) / 3600.0;

if(ageHours < 0.0)
    ageHours = 0.0;

if(reversalTarget)
{
    touch.reversalTargetIndex = targetIndex;
    touch.reversalTargetZoneID = g_zones[targetIndex].zoneID;
    touch.reversalTargetLow = g_zones[targetIndex].zoneLow;
    touch.reversalTargetHigh = g_zones[targetIndex].zoneHigh;
    touch.distanceToReversalTargetPrice = distancePrice;
    touch.distanceToReversalTargetPoints = distancePrice /
Point;
    touch.distanceToReversalTargetATR =
        touchATR > 0.0 ? distancePrice / touchATR : -1.0;
    touch.reversalTargetAgeHours = ageHours;
    touch.reversalTargetStatus = "ACTIVE";
}

```

```

    }
    else
    {
        touch.breakoutTargetIndex = targetIndex;
        touch.breakoutTargetZoneID = g_zones[targetIndex].zoneID;
        touch.breakoutTargetLow = g_zones[targetIndex].zoneLow;
        touch.breakoutTargetHigh = g_zones[targetIndex].zoneHigh;
        touch.distanceToBreakoutTargetPrice = distancePrice;
        touch.distanceToBreakoutTargetPoints = distancePrice /
Point;
        touch.distanceToBreakoutTargetATR =
            touchATR > 0.0 ? distancePrice / touchATR : -1.0;
        touch.breakoutTargetAgeHours = ageHours;
        touch.breakoutTargetStatus = "ACTIVE";
    }
}

```

//+-----

//| Открытые касания

//+-----

```

void AddOpenTouch(int touchIndex)
{
    ArrayResize(g_openTouchIndexes, g_openTouchCount + 1);
    g_openTouchIndexes[g_openTouchCount] = touchIndex;
    g_openTouchCount++;
}

```

```

void RemoveOpenTouchPosition(int position)
{
    if(position < 0 || position >= g_openTouchCount)
        return;

    for(int i = position; i < g_openTouchCount - 1; i++)
        g_openTouchIndexes[i] = g_openTouchIndexes[i + 1];

    g_openTouchCount--;
    ArrayResize(g_openTouchIndexes, g_openTouchCount);
}

```

//+-----

//| Создание записи касания

//+-----

```

void AddTouchEvent(
    int zoneIndex,
    int touchBarShift,
    datetime touchDecisionTime,

```

```

    string entryDirection
)
{
    int index = g_touchCount;
    ArrayResize(g_touches, g_touchCount + 1);
    g_touchCount++;

    ZoneData zone;
    zone = g_zones[zoneIndex];

    TouchRecord touch;

    zone.entryCount++;

    bool expectedSide = (entryDirection == "EXPECTED_SIDE");

    if(expectedSide)
    {
        zone.expectedTouchCount++;
        touch.touchNumber = zone.expectedTouchCount;
        g_expectedTouches++;
    }
    else
    {
        touch.touchNumber = 0;
        g_wrongSideEntries++;
    }

    touch.entryNumber      = zone.entryCount;
    touch.touchStatus      = expectedSide ? "TOUCH" : "REENTRY";
    touch.entryDirection = entryDirection;

    touch.zoneID          = zone.zoneID;
    touch.symbol           = zone.symbol;
    touch.timeframe        = zone.timeframe;
    touch.zoneType         = zone.zoneType;

    touch.touchBarTime     = iTime(NULL, 0, touchBarShift);
    touch.touchDecisionTime = touchDecisionTime;
    touch.touchBarShift     = touchBarShift;
    touch.touchDecisionShift = touchBarShift - 1;

    string eventPart;

    if(expectedSide)
        eventPart = "T" + IntegerToString(touch.touchNumber);
    else
        eventPart = "R" + IntegerToString(touch.entryNumber);

    touch.touchID =
        zone.zoneID + "_" + eventPart + "_" +
        TimeToID(touchDecisionTime);

```

```

double ageSeconds =
    (double) (touchDecisionTime - zone.confirmationTime);

if (ageSeconds < 0.0)
    ageSeconds = 0.0;

touch.zoneAgeCalendarMinutes = ageSeconds / 60.0;
touch.zoneAgeCalendarHours   = ageSeconds / 3600.0;

touch.zoneAgeBars =
    zone.confirmationShift - touch.touchDecisionShift;

if (touch.zoneAgeBars < 0)
    touch.zoneAgeBars = 0;

touch.zoneAgeTradingHours =
    touch.zoneAgeBars * PeriodSeconds() / 3600.0;

if (expectedSide && zone.previousExpectedTouchTime > 0)
{
    touch.minutesSincePreviousTouch =
        (double) (touchDecisionTime -
            zone.previousExpectedTouchTime) / 60.0;

    touch.barsSincePreviousTouch =
        zone.previousExpectedTouchDecisionShift -
        touch.touchDecisionShift;

    if (touch.barsSincePreviousTouch < 0)
        touch.barsSincePreviousTouch = 0;
}
else
{
    touch.minutesSincePreviousTouch = -1.0;
    touch.barsSincePreviousTouch = -1;
}

if (expectedSide)
{
    zone.previousExpectedTouchTime = touchDecisionTime;
    zone.previousExpectedTouchDecisionShift =
        touch.touchDecisionShift;
}

touch.zoneLow           = zone.zoneLow;
touch.zoneHigh          = zone.zoneHigh;
touch.zoneHeightPrice   = zone.zoneHeightPrice;
touch.zoneHeightPoints  = zone.zoneHeightPoints;
touch.zoneHeightATR     = zone.zoneHeightATR;

touch.touchATR = GetClosedATRAt(touchDecisionTime);

```

```

FillBarStats(
    touchBarShift,
    touch.touchATR,
    touch.touchBar
);

FillBarStats(
    touchBarShift + 1,
    touch.touchATR,
    touch.previousBar
);

touch.touchClose = touch.touchBar.close;

bool bullishZone = IsBullZone(zoneIndex);

if(bullishZone)
    touch.touchDepthPrice = zone.zoneHigh - touch.touchClose;
else
    touch.touchDepthPrice = touch.touchClose - zone.zoneLow;

if(touch.touchDepthPrice < 0.0)
    touch.touchDepthPrice = 0.0;

if(touch.touchDepthPrice > zone.zoneHeightPrice)
    touch.touchDepthPrice = zone.zoneHeightPrice;

touch.touchDepthPoints = touch.touchDepthPrice / Point;

if(zone.zoneHeightPrice > 0.0)
{
    touch.touchDepthPercent =
        100.0 * touch.touchDepthPrice /
        zone.zoneHeightPrice;
}
else
{
    touch.touchDepthPercent = 0.0;
}

double previousClose = touch.previousBar.close;

if(bullishZone)
    touch.approachGapPrice = previousClose - zone.zoneHigh;
else
    touch.approachGapPrice = zone.zoneLow - previousClose;

touch.approachGapPoints = touch.approachGapPrice / Point;
touch.approachGapATR =
    touch.touchATR > 0.0
    ? touch.approachGapPrice / touch.touchATR
    : -1.0;

```

```

CalculateApproachWindow(
    touchBarShift,
    3,
    bullishZone,
    touch.touchATR,
    touch.approachNetMoveATR3,
    touch.approachRangeATR3,
    touch.approachDirectionBars3
);

CalculateApproachWindow(
    touchBarShift,
    5,
    bullishZone,
    touch.touchATR,
    touch.approachNetMoveATR5,
    touch.approachRangeATR5,
    touch.approachDirectionBars5
);

CalculateApproachWindow(
    touchBarShift,
    10,
    bullishZone,
    touch.touchATR,
    touch.approachNetMoveATR10,
    touch.approachRangeATR10,
    touch.approachDirectionBars10
);

touch.reversalTargetIndex = -1;
touch.breakoutTargetIndex = -1;

int reversalTargetIndex;
int breakoutTargetIndex;

if(expectedSide)
{
    SelectStructuralTargets(
        zoneIndex,
        touchDecisionTime,
        reversalTargetIndex,
        breakoutTargetIndex
    );
}
else
{
    reversalTargetIndex = -1;
    breakoutTargetIndex = -1;
}

FillTargetData(

```

```

        reversalTargetIndex,
        zoneIndex,
        touchDecisionTime,
        touch.touchATR,
        true,
        touch
    );

FillTargetData(
    breakoutTargetIndex,
    zoneIndex,
    touchDecisionTime,
    touch.touchATR,
    false,
    touch
);

double fixedReaction = ReactionPoints * Point;
double atrReaction = touch.touchATR * ReactionATR;
double zoneReaction = zone.zoneHeightPrice *
ReactionZoneHeight;

touch.primaryReactionDistancePrice =
    MathMax(fixedReaction, MathMax(atrReaction, zoneReaction
));

touch.primaryReactionDistancePoints =
    touch.primaryReactionDistancePrice / Point;

touch.primaryReactionDistanceATR =
    touch.touchATR > 0.0
    ? touch.primaryReactionDistancePrice / touch.touchATR
    : -1.0;

touch.primaryReactionDistanceZoneHeights =
    zone.zoneHeightPrice > 0.0
    ? touch.primaryReactionDistancePrice /
zone.zoneHeightPrice
    : -1.0;

touch.localBreakoutThresholdPrice =
    MathMax(
        LocalBreakoutPoints * Point,
        touch.touchATR * LocalBreakoutATR
    );

touch.localBreakoutThresholdPoints =
    touch.localBreakoutThresholdPrice / Point;

touch.localBreakoutThresholdATR =
    touch.touchATR > 0.0
    ? touch.localBreakoutThresholdPrice / touch.touchATR

```

```

: -1.0;

touch.expectedExitReached = false;
touch.expectedExitTime = 0;
touch.barsToExpectedExit = -1;

touch.reached05ATR = false;
touch.reached10ATR = false;
touch.reached15ATR = false;
touch.reached20ATR = false;

touch.reached1ZoneHeight = false;
touch.reached2ZoneHeight = false;
touch.reached3ZoneHeight = false;

touch.primaryReversal = false;
touch.primaryReversalTime = 0;
touch.barsToPrimaryReversal = -1;

touch.localBreakout = false;
touch.localBreakoutTime = 0;
touch.barsToLocalBreakout = -1;
touch.localBreakoutClose = 0.0;
touch.localBreakoutDistancePoints = -1.0;
touch.localBreakoutDistanceATR = -1.0;

touch.localFirstResult = "";
touch.localSequence = "";

touch.structuralReversal = false;
touch.structuralReversalTime = 0;
touch.barsToStructuralReversal = -1;
touch.structuralReversalMinutes = -1.0;
touch.structuralReversalClose = 0.0;

touch.structuralBreakout = false;
touch.structuralBreakoutTime = 0;
touch.barsToStructuralBreakout = -1;
touch.structuralBreakoutMinutes = -1.0;
touch.structuralBreakoutClose = 0.0;

touch.structuralFirstResult = "";

touch.mfeClosePrice = 0.0;
touch.mfeWickPrice = 0.0;
touch.maeClosePrice = 0.0;
touch.maeWickPrice = 0.0;

touch.localObservationEndTime = 0;
touch.localObservationBars = -1;
touch.localEndReason = "";

```

```

touch.sequenceObservationEndTime = 0;
touch.sequenceObservationBars = -1;
touch.sequenceEndReason = "";

touch.structuralObservationEndTime = 0;
touch.structuralObservationBars = -1;
touch.structuralEndReason = "";

touch.localFirstDone = false;
touch.localSequenceDone = false;
touch.structuralDone = false;

if(!expectedSide)
{
    touch.localFirstResult = "NOT_EVALUATED";
    touch.localSequence = "NOT_EVALUATED";
    touch.structuralFirstResult = "NOT_EVALUATED";

    touch.localObservationEndTime = touchDecisionTime;
    touch.localObservationBars = 0;
    touch.localEndReason = "WRONG_SIDE";

    touch.sequenceObservationEndTime = touchDecisionTime;
    touch.sequenceObservationBars = 0;
    touch.sequenceEndReason = "WRONG_SIDE";

    touch.structuralObservationEndTime = touchDecisionTime;
    touch.structuralObservationBars = 0;
    touch.structuralEndReason = "WRONG_SIDE";

    touch.localFirstDone = true;
    touch.localSequenceDone = true;
    touch.structuralDone = true;

    g_zones[zoneIndex] = zone;
    g_touches[index] = touch;
    return;
}

if(reversalTargetIndex < 0 && breakoutTargetIndex < 0)
{
    touch.structuralFirstResult = "NO_TARGETS";
    touch.structuralObservationEndTime = touchDecisionTime;
    touch.structuralObservationBars = 0;
    touch.structuralEndReason = "NO_TARGETS";
    touch.structuralDone = true;
    g_structNoTargets++;
}

g_zones[zoneIndex] = zone;
g_touches[index] = touch;
AddOpenTouch(index);

```

```
}
```

```
//+-----
```

```
/// | ПОИСК НОВЫХ ВХОДОВ ЦЕНЫ В АКТИВНЫЕ ЗОНЫ
```

```
//+-----
```

```
void DetectTouchesOnBar(int closedBarShift, datetime
decisionTime)
{
    if(closedBarShift + 1 >= g_totalBars)
        return;

    double currentClose = iClose(NULL, 0, closedBarShift);
    double previousClose = iClose(NULL, 0, closedBarShift + 1);

    for(int i = 0; i < g_activeZoneCount; i++)
    {
        int zoneIndex = g_activeZoneIndexes[i];
        ZoneData zone;
        zone = g_zones[zoneIndex];

        if(zone.confirmationTime >= decisionTime)
            continue;

        if(!CloseInsideZone(currentClose, zone.zoneLow,
zone.zoneHigh))
            continue;

        string entryDirection = "";

        if(IsBullZone(zoneIndex))
        {
            if(previousClose > zone.zoneHigh)
                entryDirection = "EXPECTED_SIDE";
            else if(previousClose < zone.zoneLow)
                entryDirection = "WRONG_SIDE";
        }
        else
        {
            if(previousClose < zone.zoneLow)
                entryDirection = "EXPECTED_SIDE";
            else if(previousClose > zone.zoneHigh)
                entryDirection = "WRONG_SIDE";
        }

        // Предыдущее закрытие внутри зоны: это продолжение,
        // а не новое касание.
        if(StringLen(entryDirection) == 0)
            continue;
    }
}
```

```

        AddTouchEvent (
            zoneIndex,
            closedBarShift,
            decisionTime,
            entryDirection
        );
    }
}

```

```

//+-----

```

```

//| MFE и MAE

```

```

//+-----

```

```

void UpdateMfeMae (
    TouchRecord &touch,
    int closedBarShift
)
{
    double closePrice = iClose(NULL, 0, closedBarShift);
    double highPrice  = iHigh(NULL, 0, closedBarShift);
    double lowPrice   = iLow(NULL, 0, closedBarShift);

    double favorableClose;
    double favorableWick;
    double adverseClose;
    double adverseWick;

    if(touch.zoneType == "BULL")
    {
        favorableClose = closePrice - touch.touchClose;
        favorableWick  = highPrice - touch.touchClose;
        adverseClose   = touch.touchClose - closePrice;
        adverseWick    = touch.touchClose - lowPrice;
    }
    else
    {
        favorableClose = touch.touchClose - closePrice;
        favorableWick  = touch.touchClose - lowPrice;
        adverseClose   = closePrice - touch.touchClose;
        adverseWick    = highPrice - touch.touchClose;
    }

    if(favorableClose < 0.0)
        favorableClose = 0.0;

    if(favorableWick < 0.0)
        favorableWick = 0.0;

    if(adverseClose < 0.0)
        adverseClose = 0.0;
}

```

```

    if(adverseWick < 0.0)
        adverseWick = 0.0;

    if(favorableClose > touch.mfeClosePrice)
        touch.mfeClosePrice = favorableClose;

    if(favorableWick > touch.mfeWickPrice)
        touch.mfeWickPrice = favorableWick;

    if(adverseClose > touch.maeClosePrice)
        touch.maeClosePrice = adverseClose;

    if(adverseWick > touch.maeWickPrice)
        touch.maeWickPrice = adverseWick;
}

//+-----

//| Формирование последовательности локальных результатов

//+-----
string BuildLocalSequence(TouchRecord &touch)
{
    if(touch.primaryReversal && touch.localBreakout)
    {
        if(touch.primaryReversalTime < touch.localBreakoutTime)
            return("REVERSAL_THEN_BREAKOUT");

        if(touch.localBreakoutTime < touch.primaryReversalTime)
            return("BREAKOUT_THEN_REVERSAL");

        return("BOTH_SAME_TIME");
    }

    if(touch.primaryReversal)
        return("REVERSAL_ONLY");

    if(touch.localBreakout)
        return("BREAKOUT_ONLY");

    return("NONE");
}

//+-----

//| Обновление локального результата

//+-----
void UpdateLocalResult(
    TouchRecord &touch,

```

```

int closedBarShift,
datetime decisionTime,
int observationBars
)
{
double closePrice = iClose(NULL, 0, closedBarShift);
bool bullish = (touch.zoneType == "BULL");

double favorableFromBoundary;
double adverseBeyondBoundary;

if(bullish)
{
    favorableFromBoundary = closePrice - touch.zoneHigh;
    adverseBeyondBoundary = touch.zoneLow - closePrice;
}
else
{
    favorableFromBoundary = touch.zoneLow - closePrice;
    adverseBeyondBoundary = closePrice - touch.zoneHigh;
}

if(!touch.expectedExitReached && favorableFromBoundary > 0.0
)
{
    touch.expectedExitReached = true;
    touch.expectedExitTime = decisionTime;
    touch.barsToExpectedExit = observationBars;
}

if(touch.touchATR > 0.0)
{
    if(!touch.reached05ATR &&
        favorableFromBoundary >= 0.5 * touch.touchATR)
        touch.reached05ATR = true;

    if(!touch.reached10ATR &&
        favorableFromBoundary >= 1.0 * touch.touchATR)
        touch.reached10ATR = true;

    if(!touch.reached15ATR &&
        favorableFromBoundary >= 1.5 * touch.touchATR)
        touch.reached15ATR = true;

    if(!touch.reached20ATR &&
        favorableFromBoundary >= 2.0 * touch.touchATR)
        touch.reached20ATR = true;
}

if(!touch.reached1ZoneHeight &&
    favorableFromBoundary >= touch.zoneHeightPrice)
    touch.reached1ZoneHeight = true;

```

```

if(!touch.reached2ZoneHeight &&
    favorableFromBoundary >= 2.0 * touch.zoneHeightPrice)
    touch.reached2ZoneHeight = true;

if(!touch.reached3ZoneHeight &&
    favorableFromBoundary >= 3.0 * touch.zoneHeightPrice)
    touch.reached3ZoneHeight = true;

if(!touch.primaryReversal &&
    favorableFromBoundary >=
touch.primaryReactionDistancePrice)
{
    touch.primaryReversal = true;
    touch.primaryReversalTime = decisionTime;
    touch.barsToPrimaryReversal = observationBars;

    if(!touch.localFirstDone)
    {
        touch.localFirstResult = "REVERSAL_FIRST";
        touch.localObservationEndTime = decisionTime;
        touch.localObservationBars = observationBars;
        touch.localEndReason = "PRIMARY_REVERSAL";
        touch.localFirstDone = true;
        g_localReversalFirst++;
    }
}

if(!touch.localBreakout &&
    adverseBeyondBoundary >=
touch.localBreakoutThresholdPrice)
{
    touch.localBreakout = true;
    touch.localBreakoutTime = decisionTime;
    touch.barsToLocalBreakout = observationBars;
    touch.localBreakoutClose = closePrice;
    touch.localBreakoutDistancePoints =
        adverseBeyondBoundary / Point;
    touch.localBreakoutDistanceATR =
        touch.touchATR > 0.0
        ? adverseBeyondBoundary / touch.touchATR
        : -1.0;

    if(!touch.localFirstDone)
    {
        touch.localFirstResult = "BREAKOUT_FIRST";
        touch.localObservationEndTime = decisionTime;
        touch.localObservationBars = observationBars;
        touch.localEndReason = "LOCAL_BREAKOUT";
        touch.localFirstDone = true;
        g_localBreakoutFirst++;
    }
}

```

```

if(!touch.localFirstDone &&
    observationBars >= MaxLocalResultBars)
{
    touch.localFirstResult = "TIMEOUT";
    touch.localObservationEndTime = decisionTime;
    touch.localObservationBars = observationBars;
    touch.localEndReason = "MAX_LOCAL_BARS";
    touch.localFirstDone = true;
    g_localTimeouts++;
}

if(!touch.localSequenceDone)
{
    if((touch.primaryReversal && touch.localBreakout) ||
        observationBars >= MaxLocalResultBars)
    {
        touch.localSequence = BuildLocalSequence(touch);
        touch.sequenceObservationEndTime = decisionTime;
        touch.sequenceObservationBars = observationBars;
        touch.sequenceEndReason =
            (touch.primaryReversal && touch.localBreakout)
            ? "BOTH_RESULTS_REACHED"
            : "MAX_LOCAL_BARS";

        touch.localSequenceDone = true;
    }
}
}

```

```

//+-----
//| Обновление статуса одной целевой зоны

```

```

//+-----
void UpdateTargetStatus(
    int targetIndex,
    datetime decisionTime,
    string &targetStatus
)
{
    if(targetIndex < 0 || targetStatus != "ACTIVE")
        return;

    if(g_zones[targetIndex].brokenTime > 0 &&
        g_zones[targetIndex].brokenTime <= decisionTime)
    {
        targetStatus = "BROKEN_BEFORE_REACH";
    }
}

```

```

bool NoStructuralTargetCanBeReached(TouchRecord &touch)

```

```

{
    bool reversalPossible =
        (touch.reversalTargetIndex >= 0 &&
         touch.reversalTargetStatus == "ACTIVE");

    bool breakoutPossible =
        (touch.breakoutTargetIndex >= 0 &&
         touch.breakoutTargetStatus == "ACTIVE");

    return(!reversalPossible && !breakoutPossible);
}

//+-----+
//| Обновление структурного результата
//+-----+
void UpdateStructuralResult(
    TouchRecord &touch,
    int closedBarShift,
    datetime decisionTime,
    int observationBars
)
{
    if(touch.structuralDone)
        return;

    double closePrice = iClose(NULL, 0, closedBarShift);

    // Сначала проверяем достижение заранее зафиксированных целей.
    if(touch.reversalTargetIndex >= 0 &&
        touch.reversalTargetStatus == "ACTIVE" &&
        CloseInsideZone(
            closePrice,
            touch.reversalTargetLow,
            touch.reversalTargetHigh
        ))
    {
        touch.reversalTargetStatus = "REACHED";
        touch.structuralReversal = true;
        touch.structuralReversalTime = decisionTime;
        touch.barsToStructuralReversal = observationBars;
        touch.structuralReversalMinutes =
            (double)(decisionTime - touch.touchDecisionTime) /
60.0;
        touch.structuralReversalClose = closePrice;

        touch.structuralFirstResult = "OPPOSITE_ZONE_FIRST";
        touch.structuralObservationEndTime = decisionTime;
        touch.structuralObservationBars = observationBars;
    }
}

```

```

        touch.structuralEndReason = "OPPOSITE_ZONE_REACHED";
        touch.structuralDone = true;
        g_structReversalFirst++;
        return;
    }

    if(touch.breakoutTargetIndex >= 0 &&
        touch.breakoutTargetStatus == "ACTIVE" &&
        CloseInsideZone(
            closePrice,
            touch.breakoutTargetLow,
            touch.breakoutTargetHigh
        ))
    {
        touch.breakoutTargetStatus = "REACHED";
        touch.structuralBreakout = true;
        touch.structuralBreakoutTime = decisionTime;
        touch.barsToStructuralBreakout = observationBars;
        touch.structuralBreakoutMinutes =
60.0;
            (double)(decisionTime - touch.touchDecisionTime) /
        touch.structuralBreakoutClose = closePrice;

        touch.structuralFirstResult = "SAME_TYPE_ZONE_FIRST";
        touch.structuralObservationEndTime = decisionTime;
        touch.structuralObservationBars = observationBars;
        touch.structuralEndReason = "SAME_TYPE_ZONE_REACHED";
        touch.structuralDone = true;
        g_structBreakoutFirst++;
        return;
    }

    UpdateTargetStatus(
        touch.reversalTargetIndex,
        decisionTime,
        touch.reversalTargetStatus
    );

    UpdateTargetStatus(
        touch.breakoutTargetIndex,
        decisionTime,
        touch.breakoutTargetStatus
    );

    if(NoStructuralTargetCanBeReached(touch))
    {
        touch.structuralFirstResult = "TARGETS_BROKEN";
        touch.structuralObservationEndTime = decisionTime;
        touch.structuralObservationBars = observationBars;
        touch.structuralEndReason = "ALL_TARGETS_BROKEN";
        touch.structuralDone = true;
        g_structTargetsBroken++;
    }

```

```

        return;
    }

    if(observationBars >= MaxStructuralResultBars)
    {
        touch.structuralFirstResult = "TIMEOUT";
        touch.structuralObservationEndTime = decisionTime;
        touch.structuralObservationBars = observationBars;
        touch.structuralEndReason = "MAX_STRUCTURAL_BARS";
        touch.structuralDone = true;
        g_structTimeouts++;
    }
}

//+-----

//| Обновление всех открытых касаний

//+-----
void UpdateOpenTouchesOnBar(
    int closedBarShift,
    datetime decisionTime
)
{
    int currentDecisionShift = closedBarShift - 1;
    int position = 0;

    while(position < g_openTouchCount)
    {
        int touchIndex = g_openTouchIndexes[position];
        TouchRecord touch;
        touch = g_touches[touchIndex];

        int observationBars =
            touch.touchDecisionShift - currentDecisionShift;

        if(observationBars <= 0)
        {
            position++;
            continue;
        }

        if(observationBars <= MFEMAEObservationBars)
            UpdateMfeMae(touch, closedBarShift);

        if(!touch.localSequenceDone || !touch.localFirstDone)
        {
            if(observationBars <= MaxLocalResultBars)
            {
                UpdateLocalResult(
                    touch,

```

```

        closedBarShift,
        decisionTime,
        observationBars
    );
}
else
{
    if(!touch.localFirstDone)
    {
        touch.localFirstResult = "TIMEOUT";
        touch.localObservationEndTime = decisionTime;
        touch.localObservationBars = observationBars;
        touch.localEndReason = "MAX_LOCAL_BARS";
        touch.localFirstDone = true;
        g_localTimeouts++;
    }

    if(!touch.localSequenceDone)
    {
        touch.localSequence = BuildLocalSequence(touch);
        touch.sequenceObservationEndTime = decisionTime;
        touch.sequenceObservationBars = observationBars;
        touch.sequenceEndReason = "MAX_LOCAL_BARS";
        touch.localSequenceDone = true;
    }
}

if(!touch.structuralDone)
{
    UpdateStructuralResult(
        touch,
        closedBarShift,
        decisionTime,
        observationBars
    );
}

if(touch.localFirstDone &&
    touch.localSequenceDone &&
    touch.structuralDone &&
    observationBars >= MFEMAEObservationBars)
{
    g_touches[touchIndex] = touch;
    RemoveOpenTouchPosition(position);
    continue;
}

g_touches[touchIndex] = touch;
position++;
}
}

```

```

//+-----
//| Завершение незаконченных наблюдений в конце истории
//+-----
void FinalizeOpenTouchesAtHistoryEnd()
{
    datetime endTime = GetBarCloseTime(Period(), 1);
    int endDecisionShift = 0;

    for(int position = 0; position < g_openTouchCount; position
++)
    {
        int touchIndex = g_openTouchIndexes[position];
        TouchRecord touch;
        touch = g_touches[touchIndex];

        int observationBars =
            touch.touchDecisionShift - endDecisionShift;

        if(observationBars < 0)
            observationBars = 0;

        if(!touch.localFirstDone)
        {
            touch.localFirstResult = "END_OF_HISTORY";
            touch.localObservationEndTime = endTime;
            touch.localObservationBars = observationBars;
            touch.localEndReason = "END_OF_HISTORY";
            touch.localFirstDone = true;
        }

        if(!touch.localSequenceDone)
        {
            touch.localSequence = BuildLocalSequence(touch);
            touch.sequenceObservationEndTime = endTime;
            touch.sequenceObservationBars = observationBars;
            touch.sequenceEndReason = "END_OF_HISTORY";
            touch.localSequenceDone = true;
        }

        if(!touch.structuralDone)
        {
            touch.structuralFirstResult = "END_OF_HISTORY";
            touch.structuralObservationEndTime = endTime;
            touch.structuralObservationBars = observationBars;
            touch.structuralEndReason = "END_OF_HISTORY";
            touch.structuralDone = true;
        }

        g_touches[touchIndex] = touch;
    }
}

```

```

    }

    g_openTouchCount = 0;
    ArrayResize(g_openTouchIndexes, 0);
}

//+-----+

//| Заголовок touches.csv

//+-----+
void WriteTouchesHeader(int handle)
{
    string header =

    "TouchID;ZoneID;Symbol;Timeframe;ZoneType;EntryNumber;TouchNumber;To
    "TouchBarTime;TouchDecisionTime;ZoneAgeCalendarMinutes;ZoneAgeCalen
    "ZoneLow;ZoneHigh;ZoneHeightPrice;ZoneHeightPoints;ZoneHeightATR;To
    "TouchOpen;TouchHigh;TouchLow;TouchClosePrice;TouchDirection;TouchRa
    "PreviousOpen;PreviousHigh;PreviousLow;PreviousClose;PreviousDirect
    "ApproachGapPrice;ApproachGapPoints;ApproachGapATR;ApproachNetMoveA
    "ReversalTargetZoneID;ReversalTargetLow;ReversalTargetHigh;Distance
    "BreakoutTargetZoneID;BreakoutTargetLow;BreakoutTargetHigh;Distance
    "PrimaryReactionDistancePrice;PrimaryReactionDistancePoints;Primary
    "ExpectedExitReached;ExpectedExitTime;BarsToExpectedExit;Reached_0_5
    "PrimaryReversal;PrimaryReversalTime;BarsToPrimaryReversal;LocalBre
    "StructuralReversal;StructuralReversalTime;BarsToStructuralReversal
    "MFE_ClosePoints;MFE_CloseATR;MFE_CloseZoneHeights;MFE_WickPoints;M
    "LocalObservationEndTime;LocalObservationBars;LocalEndReason;Sequen
    ;

    FileWriteString(handle, header + "\r\n");
}

//+-----+

//| Одна строка touches.csv

```

```
//+-----
void WriteTouchRow(int handle, TouchRecord &touch)
{
    string line = "";

    AppendField(line, touch.touchID, true);
    AppendField(line, touch.zoneID);
    AppendField(line, touch.symbol);
    AppendField(line, touch.timeframe);
    AppendField(line, touch.zoneType);
    AppendField(line, IntegerToString(touch.entryNumber));
    AppendField(line, IntegerToString(touch.touchNumber));
    AppendField(line, touch.touchStatus);
    AppendField(line, touch.entryDirection);

    AppendField(line, TimeToCsv(touch.touchBarTime));
    AppendField(line, TimeToCsv(touch.touchDecisionTime));
    AppendField(line, DoubleToString(
touch.zoneAgeCalendarMinutes, 2));
    AppendField(line, DoubleToString(touch.zoneAgeCalendarHours,
6));
    AppendField(line, IntegerToString(touch.zoneAgeBars));
    AppendField(line, DoubleToString(touch.zoneAgeTradingHours,
6));
    AppendField(line, DoubleOrBlank(
touch.minutesSincePreviousTouch, 2));
    AppendField(line, IntOrBlank(touch.barsSincePreviousTouch));

    AppendField(line, PriceToString(touch.zoneLow));
    AppendField(line, PriceToString(touch.zoneHigh));
    AppendField(line, PriceToString(touch.zoneHeightPrice));
    AppendField(line, DoubleToString(touch.zoneHeightPoints, 2
));
    AppendField(line, DoubleOrBlank(touch.zoneHeightATR, 6));
    AppendField(line, PriceToString(touch.touchATR));
    AppendField(line, PriceToString(touch.touchClose));
    AppendField(line, PriceToString(touch.touchDepthPrice));
    AppendField(line, DoubleToString(touch.touchDepthPoints, 2
));
    AppendField(line, DoubleToString(touch.touchDepthPercent, 4
));

    AppendField(line, PriceToString(touch.touchBar.open));
    AppendField(line, PriceToString(touch.touchBar.high));
    AppendField(line, PriceToString(touch.touchBar.low));
    AppendField(line, PriceToString(touch.touchBar.close));
    AppendField(line, touch.touchBar.direction);
    AppendField(line, DoubleOrBlank(touch.touchBar.rangeATR, 6
));
    AppendField(line, DoubleOrBlank(touch.touchBar.bodyATR, 6));
}
```

```

AppendField(line, DoubleToString(touch.touchBar.bodyPercent,
4));
AppendField(line, DoubleToString(
touch.touchBar.upperWickPercent, 4));
AppendField(line, DoubleToString(
touch.touchBar.lowerWickPercent, 4));
AppendField(line, LongToCsv(touch.touchBar.tickVolume));
AppendField(line, DoubleOrBlank(
touch.touchBar.relativeVolume, 6));

AppendField(line, PriceToString(touch.previousBar.open));
AppendField(line, PriceToString(touch.previousBar.high));
AppendField(line, PriceToString(touch.previousBar.low));
AppendField(line, PriceToString(touch.previousBar.close));
AppendField(line, touch.previousBar.direction);
AppendField(line, DoubleOrBlank(touch.previousBar.rangeATR,
6));
AppendField(line, DoubleOrBlank(touch.previousBar.bodyATR, 6
));
AppendField(line, DoubleToString(
touch.previousBar.bodyPercent, 4));
AppendField(line, LongToCsv(touch.previousBar.tickVolume));
AppendField(line, DoubleOrBlank(
touch.previousBar.relativeVolume, 6));

AppendField(line, PriceToString(touch.approachGapPrice));
AppendField(line, DoubleToString(touch.approachGapPoints, 2
));
AppendField(
    line,
    touch.touchATR > 0.0
    ? DoubleToString(touch.approachGapATR, 6)
    : ""
);
AppendField(line, DoubleToString(touch.approachNetMoveATR3,
6));
AppendField(line, DoubleToString(touch.approachNetMoveATR5,
6));
AppendField(line, DoubleToString(touch.approachNetMoveATR10,
6));
AppendField(line, DoubleOrBlank(touch.approachRangeATR3, 6
));
AppendField(line, DoubleOrBlank(touch.approachRangeATR5, 6
));
AppendField(line, DoubleOrBlank(touch.approachRangeATR10, 6
));
AppendField(line, IntOrBlank(touch.approachDirectionBars3));
AppendField(line, IntOrBlank(touch.approachDirectionBars5));
AppendField(line, IntOrBlank(touch.approachDirectionBars10
));

AppendField(line, touch.reversalTargetZoneID);

```

```

        AppendField(line, PriceOrBlank(touch.reversalTargetLow));
        AppendField(line, PriceOrBlank(touch.reversalTargetHigh));
        AppendField(line, PriceOrBlank(
touch.distanceToReversalTargetPrice));
        AppendField(line, DoubleOrBlank(
touch.distanceToReversalTargetPoints, 2));
        AppendField(line, DoubleOrBlank(
touch.distanceToReversalTargetATR, 6));
        AppendField(line, DoubleOrBlank(touch.reversalTargetAgeHours
, 6));
        AppendField(line, touch.reversalTargetStatus);

        AppendField(line, touch.breakoutTargetZoneID);
        AppendField(line, PriceOrBlank(touch.breakoutTargetLow));
        AppendField(line, PriceOrBlank(touch.breakoutTargetHigh));
        AppendField(line, PriceOrBlank(
touch.distanceToBreakoutTargetPrice));
        AppendField(line, DoubleOrBlank(
touch.distanceToBreakoutTargetPoints, 2));
        AppendField(line, DoubleOrBlank(
touch.distanceToBreakoutTargetATR, 6));
        AppendField(line, DoubleOrBlank(touch.breakoutTargetAgeHours
, 6));
        AppendField(line, touch.breakoutTargetStatus);

        AppendField(line, PriceToString(
touch.primaryReactionDistancePrice));
        AppendField(line, DoubleToString(
touch.primaryReactionDistancePoints, 2));
        AppendField(line, DoubleOrBlank(
touch.primaryReactionDistanceATR, 6));
        AppendField(line, DoubleOrBlank(
touch.primaryReactionDistanceZoneHeights, 6));
        AppendField(line, PriceToString(
touch.localBreakoutThresholdPrice));
        AppendField(line, DoubleToString(
touch.localBreakoutThresholdPoints, 2));
        AppendField(line, DoubleOrBlank(
touch.localBreakoutThresholdATR, 6));

        AppendField(line, BoolToCsv(touch.expectedExitReached));
        AppendField(line, TimeToCsv(touch.expectedExitTime));
        AppendField(line, IntOrBlank(touch.barsToExpectedExit));
        AppendField(line, BoolToCsv(touch.reached05ATR));
        AppendField(line, BoolToCsv(touch.reached10ATR));
        AppendField(line, BoolToCsv(touch.reached15ATR));
        AppendField(line, BoolToCsv(touch.reached20ATR));
        AppendField(line, BoolToCsv(touch.reached1ZoneHeight));
        AppendField(line, BoolToCsv(touch.reached2ZoneHeight));
        AppendField(line, BoolToCsv(touch.reached3ZoneHeight));

        AppendField(line, BoolToCsv(touch.primaryReversal));

```

```

AppendField(line, TimeToCsv(touch.primaryReversalTime));
AppendField(line, IntOrBlank(touch.barsToPrimaryReversal));
AppendField(line, BoolToCsv(touch.localBreakout));
AppendField(line, TimeToCsv(touch.localBreakoutTime));
AppendField(line, IntOrBlank(touch.barsToLocalBreakout));
AppendField(
    line,
    touch.localBreakout ? PriceToString(
touch.localBreakoutClose) : ""
);
AppendField(line, DoubleOrBlank(
touch.localBreakoutDistancePoints, 2));
AppendField(line, DoubleOrBlank(
touch.localBreakoutDistanceATR, 6));
AppendField(line, touch.localFirstResult);
AppendField(line, touch.localSequence);

AppendField(line, BoolToCsv(touch.structuralReversal));
AppendField(line, TimeToCsv(touch.structuralReversalTime));
AppendField(line, IntOrBlank(touch.barsToStructuralReversal
));
AppendField(line, DoubleOrBlank(
touch.structuralReversalMinutes, 2));
AppendField(
    line,
    touch.structuralReversal
    ? PriceToString(touch.structuralReversalClose)
    : ""
);

AppendField(line, BoolToCsv(touch.structuralBreakout));
AppendField(line, TimeToCsv(touch.structuralBreakoutTime));
AppendField(line, IntOrBlank(touch.barsToStructuralBreakout
));
AppendField(line, DoubleOrBlank(
touch.structuralBreakoutMinutes, 2));
AppendField(
    line,
    touch.structuralBreakout
    ? PriceToString(touch.structuralBreakoutClose)
    : ""
);
AppendField(line, touch.structuralFirstResult);

double mfeClosePoints = touch.mfeClosePrice / Point;
double mfeWickPoints = touch.mfeWickPrice / Point;
double maeClosePoints = touch.maeClosePrice / Point;
double maeWickPoints = touch.maeWickPrice / Point;

double mfeCloseATR =
    touch.touchATR > 0.0 ? touch.mfeClosePrice /
touch.touchATR : -1.0;

```

```

    double mfeWickATR =
        touch.touchATR > 0.0 ? touch.mfeWickPrice /
touch.touchATR : -1.0;
    double maeCloseATR =
        touch.touchATR > 0.0 ? touch.maeClosePrice /
touch.touchATR : -1.0;
    double maeWickATR =
        touch.touchATR > 0.0 ? touch.maeWickPrice /
touch.touchATR : -1.0;

    double mfeCloseHeights =
        touch.zoneHeightPrice > 0.0
        ? touch.mfeClosePrice / touch.zoneHeightPrice : -1.0;
    double mfeWickHeights =
        touch.zoneHeightPrice > 0.0
        ? touch.mfeWickPrice / touch.zoneHeightPrice : -1.0;
    double maeCloseHeights =
        touch.zoneHeightPrice > 0.0
        ? touch.maeClosePrice / touch.zoneHeightPrice : -1.0;
    double maeWickHeights =
        touch.zoneHeightPrice > 0.0
        ? touch.maeWickPrice / touch.zoneHeightPrice : -1.0;

    AppendField(line, DoubleToString(mfeClosePoints, 2));
    AppendField(line, DoubleOrBlank(mfeCloseATR, 6));
    AppendField(line, DoubleOrBlank(mfeCloseHeights, 6));
    AppendField(line, DoubleToString(mfeWickPoints, 2));
    AppendField(line, DoubleOrBlank(mfeWickATR, 6));
    AppendField(line, DoubleOrBlank(mfeWickHeights, 6));
    AppendField(line, DoubleToString(maeClosePoints, 2));
    AppendField(line, DoubleOrBlank(maeCloseATR, 6));
    AppendField(line, DoubleOrBlank(maeCloseHeights, 6));
    AppendField(line, DoubleToString(maeWickPoints, 2));
    AppendField(line, DoubleOrBlank(maeWickATR, 6));
    AppendField(line, DoubleOrBlank(maeWickHeights, 6));

    AppendField(line, TimeToCsv(touch.localObservationEndTime));
    AppendField(line, IntOrBlank(touch.localObservationBars));
    AppendField(line, touch.localEndReason);
    AppendField(line, TimeToCsv(touch.sequenceObservationEndTime
));
    AppendField(line, IntOrBlank(touch.sequenceObservationBars
));
    AppendField(line, touch.sequenceEndReason);
    AppendField(line, TimeToCsv(
touch.structuralObservationEndTime));
    AppendField(line, IntOrBlank(touch.structuralObservationBars
));
    AppendField(line, touch.structuralEndReason);

    FileWriteString(handle, line + "\r\n");
}

```

```

//+-----
//| Сохранение touches.csv
//+-----
bool SaveTouchesCsv()
{
    ResetLastError();

    int handle = FileOpen(
        OutputTouchesFileName,
        FILE_WRITE | FILE_TXT | FILE_ANSI
    );

    if(handle == INVALID_HANDLE)
    {
        Print(
            "Ошибка создания ",
            OutputTouchesFileName,
            ". Код ошибки: ",
            GetLastError()
        );

        ResetLastError();
        return(false);
    }

    WriteTouchesHeader(handle);

    for(int i = 0; i < g_touchCount; i++)
        WriteTouchRow(handle, g_touches[i]);

    FileFlush(handle);
    FileClose(handle);

    string fullPath =
        TerminalInfoString(TERMINAL_DATA_PATH) +
        "\\MQL4\\Files\\" +
        OutputTouchesFileName;

    Print("Файл создан: ", fullPath);
    return(true);
}

//+-----
//| Проверка параметров
//+-----
bool ValidateInputs()

```

```

{
    if(StringLen(InputZonesFileName) == 0 ||
        StringLen(OutputTouchesFileName) == 0)
    {
        Print("Ошибка: имена файлов не могут быть пустыми.");
        return(false);
    }

    if(TouchATRPeriod < 1)
    {
        Print("Ошибка: TouchATRPeriod должен быть больше 0.");
        return(false);
    }

    if(RelativeVolumePeriod < 1)
    {
        Print(
"Ошибка: RelativeVolumePeriod должен быть больше 0.");
        return(false);
    }

    if(ReactionPoints < 0.0 ||
        ReactionATR < 0.0 ||
        ReactionZoneHeight < 0.0)
    {
        Print(
"Ошибка: параметры разворота не могут быть отрицательными.");
        return(false);
    }

    if(LocalBreakoutPoints < 0.0 ||
        LocalBreakoutATR < 0.0)
    {
        Print(
"Ошибка: параметры локального пробоя не могут быть отрицательными."
);
        return(false);
    }

    if(MaxLocalResultBars < 1 ||
        MaxStructuralResultBars < 1 ||
        MFEMAEObservationBars < 1)
    {
        Print("Ошибка: окна наблюдения должны быть больше 0.");
        return(false);
    }

    if(ProgressEveryBars < 0)
    {
        Print(
"Ошибка: ProgressEveryBars не может быть отрицательным.");
        return(false);
    }
}

```

```

    return(true);
}

//+-----

//| Главная функция

//+-----
void OnStart()
{
    if(!ValidateInputs())
        return;

    g_totalBars = iBars(NULL, 0);

    if(g_totalBars < 20)
    {
        Print("Недостаточно истории на графике: ", g_totalBars,
" баров.");
        return;
    }

    g_touchAttrTf = (int)TouchATRTimeframe;

    if(g_touchAttrTf == PERIOD_CURRENT || g_touchAttrTf == 0)
        g_touchAttrTf = Period();

    ArrayResize(g_zones, 0);
    ArrayResize(g_touches, 0);
    ArrayResize(g_activeZoneIndexes, 0);
    ArrayResize(g_openTouchIndexes, 0);

    g_zoneCount = 0;
    g_touchCount = 0;
    g_activeZoneCount = 0;
    g_openTouchCount = 0;
    g_nextZoneToActivate = 0;

    g_loadedRows = 0;
    g_skippedRejected = 0;
    g_skippedOtherChart = 0;
    g_expectedTouches = 0;
    g_wrongSideEntries = 0;
    g_localReversalFirst = 0;
    g_localBreakoutFirst = 0;
    g_localTimeouts = 0;
    g_structReversalFirst = 0;
    g_structBreakoutFirst = 0;
    g_structNoTargets = 0;
    g_structTargetsBroken = 0;
    g_structTimeouts = 0;

```

```

    Print(
"=====
");
);
    Print("RectangleZoneAnalyzer_02_v3");
    Print("Символ: ", Symbol(), " ТФ: ", TimeframeToString(
Period()));
    Print("Чтение файла: ", InputZonesFileName);
    Print(
"=====
");

    if(!LoadZonesCsv())
        return;

    int firstClosedBarShift =
        g_zones[0].confirmationShift;

    if(firstClosedBarShift > g_totalBars - 2)
        firstClosedBarShift = g_totalBars - 2;

    if(firstClosedBarShift < 1)
    {
        Print("Нет баров после первой подтверждённой зоны.");
        return;
    }

    int totalBarsToProcess = firstClosedBarShift;
    int processed = 0;

    for(int closedBarShift = firstClosedBarShift;
        closedBarShift >= 1;
        closedBarShift--)
    {
        if(IsStopped())
        {
            Print("Выполнение остановлено пользователем.");
            return;
        }

        datetime decisionTime =
            GetBarCloseTime(Period(), closedBarShift);

        if(decisionTime <= 0)
        {
            Print("Ошибка времени закрытия бара shift=",
closedBarShift);
            return;
        }

        // 1. Активируем только зоны, уже известные до этого закрытия.
        ActivateKnownZones(decisionTime);
    }

```

```

// 2. Обновляем результаты ранее зарегистрированных касаний.
UpdateOpenTouchesOnBar(
    closedBarShift,
    decisionTime
);

// 3. Удаляем зоны, пробитые текущим закрытием.
RemoveBrokenZones(decisionTime);

// 4. Ищем новые закрытия внутри оставшихся активных зон.
DetectTouchesOnBar(
    closedBarShift,
    decisionTime
);

processed++;

if(ProgressEveryBars > 0 &&
    (processed % ProgressEveryBars == 0 ||
     processed == totalBarsToProcess))
{
    double percent =
        100.0 * processed / totalBarsToProcess;

    Print(
        "Прогресс: ",
        processed,
        "/",
        totalBarsToProcess,
        " (",
        DoubleToString(percent, 1),
        "%)",
        "    активных зон=",
        g_activeZoneCount,
        "    касаний=",
        g_touchCount,
        "    наблюдений=",
        g_openTouchCount
    );
}
}

FinalizeOpenTouchesAtHistoryEnd();

bool saved = SaveTouchesCsv();

Print(
"=====
");

Print("РЕЗУЛЬТАТ АНАЛИЗА КАСАНИЙ");
Print("Строк прочитано zones.csv : ", g_loadedRows);
Print("Принятых зон загружено      : ", g_zoneCount);

```

```

Print("Отклонённых пропущено      : ", g_skippedRejected);
Print("Другой символ/ТФ пропущено: ", g_skippedOtherChart);
Print("Правильных касаний          : ", g_expectedTouches);
Print("Входов с обратной стороны  : ", g_wrongSideEntries);
Print("Всего строк touches.csv     : ", g_touchCount);
Print("Локально: разворот первым   : ", g_localReversalFirst);
Print("Локально: пробой первым     : ", g_localBreakoutFirst);
Print("Локально: timeout            : ", g_localTimeouts);
Print("Структурно: против. зона     : ", g_structReversalFirst
);
Print("Структурно: одностип. зона  : ", g_structBreakoutFirst
);
Print("Структурно: целей нет        : ", g_structNoTargets);
Print("Структурно: цели пробиты     : ", g_structTargetsBroken
);
Print("Структурно: timeout          : ", g_structTimeouts);
Print("Сохранение файла             : ", saved ? "OK" : "ERROR"
);
Print(
"=====
);
}

```

//+-----